Design Report

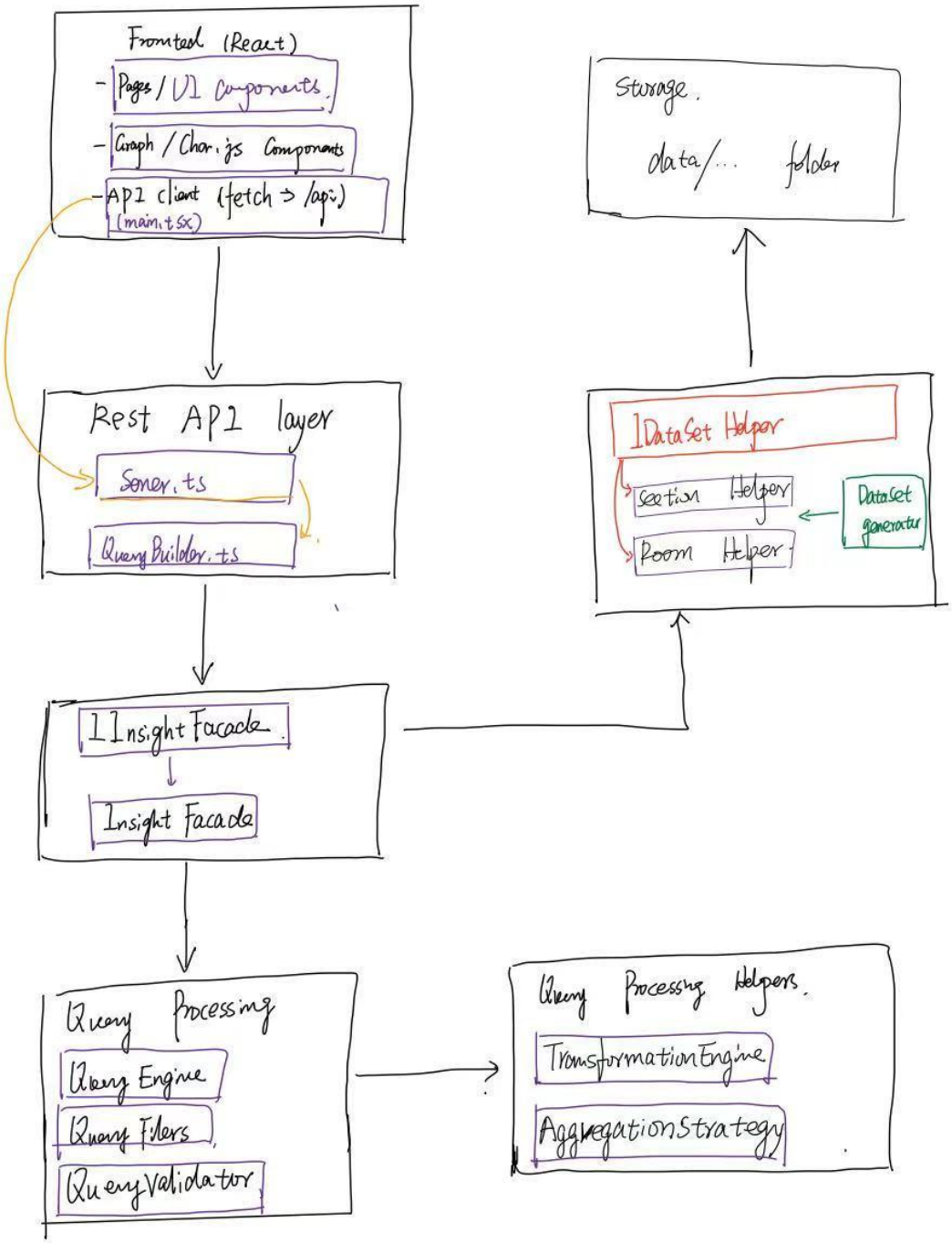
Team Repository: project_team103

Specification: Section Insights

Project Video Link: https://youtu.be/9-n3MyP81c8

User Stories Link: https://github.students.cs.ubc.ca/CPSC310-2025W-T1/project_team103/issues/20#issue-177252

Part A: For Engineers



The graph above is our architecture graph with different components

1. Design Strength
    a. Clear Separation of Concerns
        i. Frontend UI only handles presentation logic
        ii. REST API Layer (Server.ts) only routes HTTP requests
        iii. QueryBuilder.ts encapsulates query construction by removing query-building logic from the server, which can improve readability.
        iv. InsightFacade follows the Facade Pattern, which only exposes entry points for dataset operations and hides the rest of the system.
        v. The Query Processing Engine part isolates the Query logic, which has high cohesion.
    b. Benefit:

        Our design follows the Single Responsibility Principle and keeps each layer loosely coupled. It improves debugging and future maintenance of the whole system.
2. Design Weakness
    a. High Coupling Between Frontend and Custom Analytics Endpoints
        i. The frontend relies heavily on specific backend routes such as:

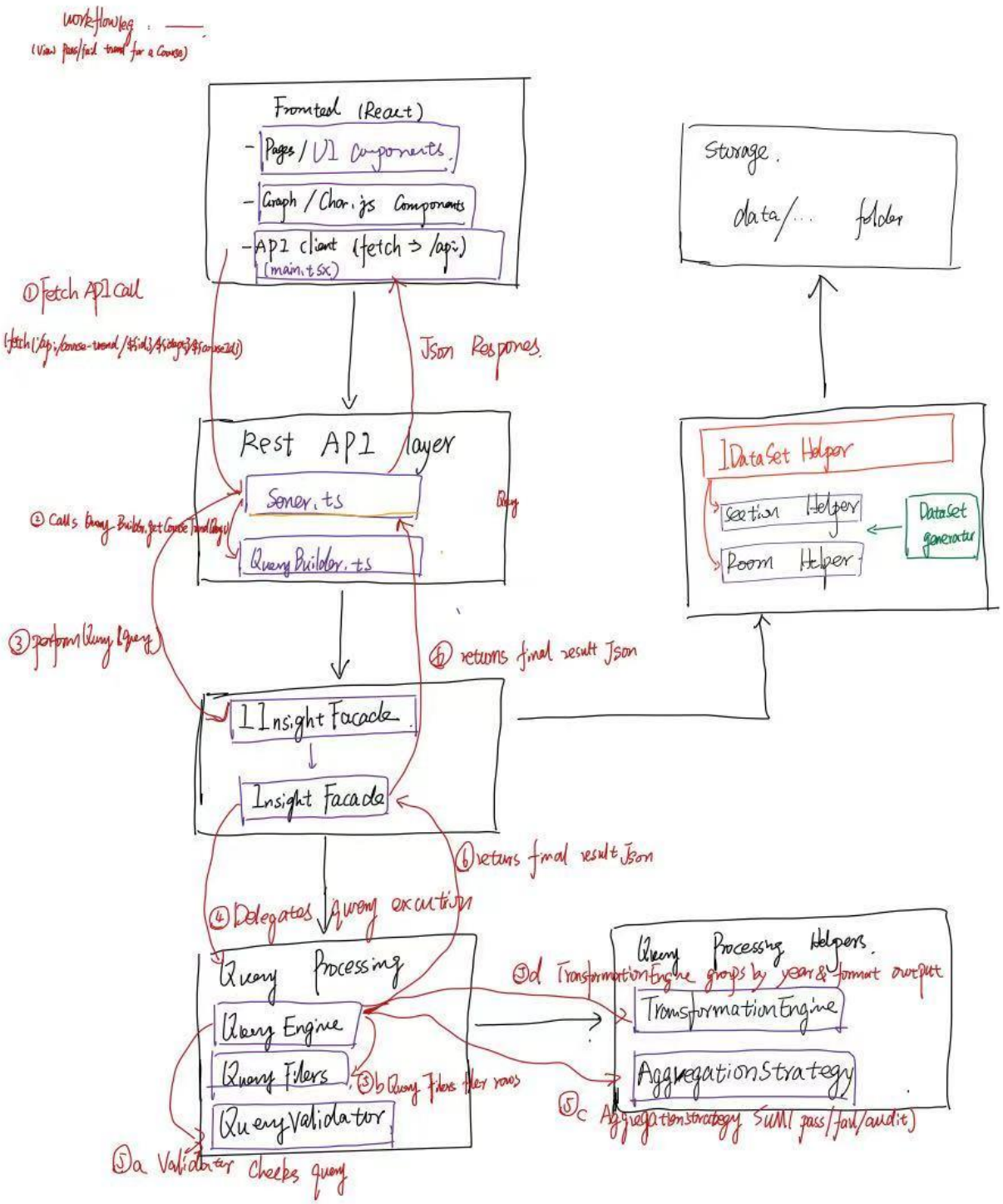
            /api/course-trend/:id/:dept/:courseId

            /api/course-instructors/:id/:dept/:courseId

            /api/instructor-courses/:datasetId/:instructorName

            1. This creates tight coupling between the UI and backend layer.

            2. If the backend changes its analytics API structure or renames endpoints, the frontend must be updated in multiple places.
        ii. We chose so many different and specific endpoints just because we want to simplify frontend logic and reduce query authoring complexity. We want to complete all these logics in the backend.
3. End-to-End Workflow (add all the labeled paths in the architecture graph): e.g. Viewing Pass/Fail Trend for a Course



The red numbered arrows on the architecture diagram illustrate the end-to-end workflow for the course trend visualization:

1. **Frontend (React)**

   The user selects a dataset, department, and course. The UI issues a GET request to /api/course-trend/{dataset}/{dept}/{courseId}.
2. **REST API Layer (Server.ts)**

   Server.ts receives the request, extracts parameters, and calls QueryBuilder.getCourseTrendQuery(id, dept, courseId) which constructs the InsightUBC query JSON.
3. **InsightFacade.performQuery(query)**

   Server.ts forwards the constructed query to InsightFacade, which acts as a facade and delegates the work internally.
4. **Query Processing Engine**

   The query is executed through several coordinated components:
   - **5a QueryValidator** checks the structure of the query.
   - **5b QueryFilters** filters rows for the given dept, course, and valid years.
   - **5c AggregationStrategy** computes SUM (pass), SUM (fail), and SUM (audit) for each year.
   - **5d TransformationEngine** groups results by year and formats the transformation output.
5. **Result Formatting**

   InsightFacade calls QueryBuilder.formatTrendData() to convert raw query results into the final version:

   {year, pass, fail, audit, passRate, total}

   sorted by year.
6. **REST API Response**

   Server.ts sends the final JSON back to the frontend.
7. **Frontend Rendering**

   The visualisation layer renders a line chart using the returned array, completing the workflow.

4. Alternative Design- Frontend Builds InsightUBC Queries Directly:

Instead of exposing multiple specialised analytics endpoints (e.g. /api/course-trend/..., /api/instructors/...), the frontend could construct a single InsightUBC query api and call only: POST /api/query.

In this alternative design:

- The frontend needs to be responsible for building full InsightUBC query JSON for filters.
- The backend only exposes the generic /api/query endpoint without QueryBuilder helpers.
- All analytics logic needs to be implemented on the frontend rather than the backend, since we return raw data directly from the backend.

Comparison of the Two Designs:

Tradeoff (Why not choose?):

1. Increase Frontend Complexity:
    a. InsightUBC query syntax is too complex and nested, if the frontend had to manually construct all the queries, the UI code would become significantly harder to maintain.
    b. Our current backend-driven design reduces logics on the frontend and keeps it only focused on one rendering thing (SRP).
2. Poor Testability on the Frontend:
    a. After transferring analytics logic into React, we lost the ability to unit-test queries in isolation and mock dataset transformations.
    b. But the current design places all query-building logic in the QueryBuilder in backend, which enables unit tests for each query shape and for analytics logic across all UI components

Why choose the current design?

1. Separation of Tasks:

All analytics logic, such as QueryBuilder and Course trend/instructor summaries, stays in the backend, and the frontend only displays filtered data, which follows SRP.

2. More Maintainable and Testable Architecture:

   Backend query logic and code can do the integration/unit tests for the query and. can easily modify or extend without changing the Frontend's code/structure.

Summary: Although letting the frontend construct InsightUBC queries is the general way to solve this problem, we chose to put all the query-building design encapsulated in the backend. Because this design can reduce frontend complexity, improve testability, and maintain lower coupling between UI and data semantics. These tradeoffs make the whole project easier to maintain and evolve.

- REST API Design

Dataset Management		
URL	**Verb**	**Action**
/api/dataset/:id	PUT	Upload a new dataset of Base64 ZIP
/api/dataset/:id	DELETE	Remove an existing dataset
/api/datasets	GET	List all loaded datasets
Analytics Endpoints		
/api/departments/:id	GET	Get all departments in a dataset
/api/courses/:id/:dept	GET	Get all data within slected department
/api/course-trend/:id/:dept/:courseId	GET	Get pass/fail/audit data for the selected course
/api/course-instructors/:id/:dept/:courseId	GET	Get instructors who taught the course and their statistics
/api/instructors/:id/:dept	GET	Get all instructors who taught within a department
/api/instructor-courses/:datasetId/:instructorName	GET	Get all courses and its statistics taught by a specific instructor

This Table above includes all the rest API we use

a. Why this decomposition?

   We deliberately designed our REST API as a set of domain-specific endpoints rather than a single general /api/query entry point.

   This decomposition supports our frontend workflow and keeps all analytics logic only inside the backend.

   i. Benefit 1: High Cohesion Per Endpoint

      In our design, each endpoint returns one specific dataset, which is sorted by departments, courses or instructors. This avoids endpoints with many

tasks and purposes and keeps the API easy to understand for future developers.

ii. Benefit 2: More Stable and Safe Public API

Frontend depends only on simple REST routes. If InsightUBC's internal query syntax changes, the backend hides this change behind stable endpoints. And our REST API can also avoid directly sending the unsorted data to the frontend to make the frontend safe.

Part B: For Managers

1. Testing Quality Evaluation

```
  Server API Integration Tests (backend)
    ✔ PUT /api/dataset/:id should add dataset successfully
    ✔ GET /api/datasets should list datasets
    ✔ DELETE /api/dataset/:id should remove dataset successfully
    ✔ PUT /api/dataset/:id should re-add dataset for query tests
    ✔ GET /api/departments/:id should return list of departments
    ✔ GET /api/courses/:id/:dept should return list of courses in department
    ✔ GET /api/course-trend/:id/:dept/:courseId should return pass/fail trend for a course
    ✔ GET /api/course-instructors/:id/:dept/:courseId should return instructors with grades for a course
    ✔ GET /api/instructors/:id/:dept should return list of unique instructors in department
    ✔ GET /api/instructors/:id/:dept should fail for non-existent dataset
    ✔ GET /api/instructor-courses/:datasetId/:instructorName should return courses taught by instructor
    ✔ GET /api/instructor-courses/:datasetId/:instructorName should fail for non-existent dataset


  135 passing (4s)

-------------------------|---------|----------|---------|---------|-------------------------------------------------------------
File                     | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------------|---------|----------|---------|---------|-------------------------------------------------------------
All files                |   92.26 |    79.93 |    95.4 |   91.98 |
 controller              |   93.29 |     83.9 |   98.24 |   93.02 |
  AggregationStrategy.ts |   97.05 |        0 |     100 |   96.66 | 60
  DatasetGenerator.ts    |    87.5 |    66.66 |     100 |    87.5 | 14
  IInsightFacade.ts      |     100 |      100 |     100 |     100 |
  InsightFacade.ts       |   96.72 |      100 |     100 |   96.55 | 103,126
  QueryEngine.ts         |   86.79 |     77.5 |   94.44 |    86.4 | 35,58,92,113,154-164,179
  QueryFilters.ts        |   97.64 |    95.55 |     100 |   97.59 | 94,109
  QueryValidator.ts      |    87.5 |    84.04 |     100 |   87.28 | 83,92,110,120,124,132,152,157,162,178,183,202,243,247,252
  RoomHelper.ts          |    95.1 |    75.92 |   96.66 |   94.77 | 21,66,81,103-111,221
  SectionHelper.ts       |     100 |       90 |     100 |     100 | 49
  TransformationEngine.ts |    100 |      100 |     100 |     100 |
 rest                    |      88 |    52.63 |      90 |   87.75 |
  QueryBuilder.ts        |     100 |       75 |     100 |     100 | 70,74-111,179
  Server.ts              |      85 |    27.77 |   83.78 |   84.74 | 63,80,96,112,147-209,218,230,244,260
-------------------------|---------|----------|---------|---------|-------------------------------------------------------------
```

The graph above shows the statistical data (tests & coverage) of our REST API integration test. We test our backend system using both integration and unit tests for all query-processing components. And we can see all of our tests have successfully passed, and our overall code coverage is statements**:** 92.26%, branches**:** 79.93%, functions**:**

95.4%, and lines: 91.98%. This indicates that most of our logic paths and data-processing steps have been tested.

Test Coverage:

We covered all dataset management routes (PUT/api/dataset, DELETE /api/dataset, GET /api/datasets). And we also tested all analytics endpoints, including departments, courses, course-trend, course-instructors, and instructors. In addition, we also test the entire performQuery through these integration tests, which ensures that all operations of query processing can run correctly on real data.

Observability:

Our API returns structured JSON ({result: ...} or {error: ...}), so when a test. fails/succeeds, the message is immediately visible. This improves diagnosability because the test output clearly compares the expected JSON with the actual one.

Areas to Improve

First, some branches in Server.ts were not covered, so we need to consider more. parameters and logic tests to improve branch coverage. And we need to add more edge-case tests, such as empty departments, instructor names with semicolons and so on. In addition, we also need to add more tests for QueryBuilder, and that would help us ensure the correctness of complex query-building logic.

2. Why these four user stories? Future adaptation?

We selected these 4 stories because they can work together to complete the core. Functionality and interaction workflow of the InsightUBC application. After the completion of the 4 user stories, (1) add data, (2) remove data, (3) view available datasets, and (4) generate insights from those datasets, the application can support most of the usage scenarios and user demands. Therefore, these stories provide the highest value from both user and system design, with a minimal, workable workflow.

- User Story 1 (Add Dataset) and User Story 2 (Remove Dataset) are the core parts of. the dataset. management and is also the foundation of the entire system. If we cannot complete the upload or delete datasets functionality, users may encounter some errors in further analysis of datasets.
- User Story 3 (View All Datasets) is necessary because it is the bridge between. dataset management and data analysis. Users need to be able to see which datasets exist with their sizes and type, so that users can decide to generate Insights.
- User Story 4 (View Visual Insights) is the key part of the whole application, as it can help students understand course patterns, instructor performance, and grade trends.

Possible Future Change for User Story 1 & 2

In the future, dataset management may become more advanced, for example, for users. may want to upload multiple datasets at once or upload datasets in different formats (CSV/JSON, not only ZIP). In addition, users may want to delete all added datasets at once rather than one at a time, and have the option to restore data later.

Required Design Adaptation

To support these changes, we would need to extend the dataset management part. First, addDataset() would need to support mass upload and add extra parsers for each dataset type with specific version metadata. Then, deleteDataset() would support one-time mass deletion operations by generalising REST endpoints (e.g., /api/datasets/mass). Our current design already separates dataset management in InsightFacade from routing Server.ts, so these future changes can be added with simple extensions rather than major redesigns and refactoring.

2. Security Vulnerability

   Potential Security Vulnerability:

Malicious or malformed dataset uploads may be possible in our application. Security vulnerability. Any user can upload arbitrary ZIP files without any limit, so if a user maliciously uploads an extremely large or corrupted ZIP file to cause excessive CPU/ memory usage during parsing, the whole application may run out of storage and crash.

Handle:

Our system already performs dataset type validation checking and rejects files. Which is not ZIP type. To further mitigate the risk, we can enforce file-size limits, set a rate limit on upload requests when users add new datasets, and perform ZIP integrity checks before extraction. In addition, we need to add extra try/catch blocks to check parsing errors and prevent server crashes. Therefore, these measures ensure the backend cannot be easily overloaded or crashed by harmful uploads.

3. Ethical Situation (RGB format)

Situation: Our application displays instructor performance statistics by using. historical grade trends. This is useful for students to make course&instructor selection, but this can also influence instructors' course enrollment and even affect students' attitude to instructors.

Role: If I am an Instructors in UBC, I am in the primary group affected in this. situation.

Goal: As an instructor, I want to be fairly represented and evaluated based on the full context of my teaching including course difficulty, class size and term variations, rather than directly use raw grade averages alone to evaluate my teaching level.

Benefits: The application promotes transparency in course information. Students gain better insight into historical grade trends, helping them make informed academic decisions. And this can benefit instructors whose courses are consistently well received.

Drawback:

Because we present grade averages and instructor statistics without contextual factors (e.g., difficulty, mandatory courses, enrollment differences), some instructors may be unfairly judged by students. This can lead to bad enrollment patterns and reputational damage for instructors who teach more challenging courses， which is an ethical issue as the system emphasises one factor (grades) while ignoring the abundant teaching context.